



Master of Aerospace Engineering – Embedded Systems

Research Project Final Report

A Fault Injection Framework for Systematic Evaluation of RISC-V Fault Mitigations in Space

Authors: Szymon Bogus
Raj Jayesh-Popat
Malte von Ehren

Tutors: Arnaud Dion
Michaël Lauer

Submission Date: 26th of March 2026

We hereby confirm that this report is our own work and we have documented all sources and material used.

A Fault Injection Framework for Systematic Evaluation of RISC-V Fault Mitigations in Space

Szymon Bogus

Raj Jayesh-Popat

Malte von Ehren

Abstract—Radiation effects such as single-event upsets (SEUs) pose significant reliability challenges for processors in space environments, motivating the development of fault-tolerant designs. While extensive research efforts have proposed mitigation techniques for different cores, the challenge of comparing their effectiveness remains difficult due to the lack of a common evaluation infrastructure. This paper presents a modular framework for systematic fault-injection campaigns and comparative analysis of processor cores under SEUs. While applicable to multiple architectures, the framework is demonstrated using RISC-V due to its open-standard and growing interest in space applications. The framework enables interchangeable integration of processor implementations and automated analysis of fault-injection results. To demonstrate its applicability, the framework was validated and evaluated on a custom RISC-V core, enabling the assessment of mitigation techniques such as Triple Modular Redundancy (TMR), Error Correction Codes (ECC), and the removal of speculative execution.

Index Terms—Fault Injection, RISC-V, FPGAs, Single-Event Upsets (SEUs), Radiation Effects, Processor Reliability

I. INTRODUCTION

With the growing interest in space exploration and satellite technology, the reliability of computing systems in harsh space environments has become a critical concern. Space radiation can lead to various faults in microprocessors, which, if not properly mitigated, may compromise the success of a mission. Space-grade processors rely on a mixture of radiation-hardened materials, hardware, and software protection. Many currently utilized processors are years behind modern commercial alternatives [1], [2]. They are costly and offer limited flexibility for contemporary software solutions.

Among emerging alternatives, RISC-V has attracted increasing attention because its open and modular architecture enables customizable and efficient fault-tolerant designs [3], [4]. However, despite many proposed mitigation techniques, comparing their effectiveness across cores remains difficult because a common evaluation infrastructure is often missing [5]. This paper addresses that gap with a modular framework for systematic fault-injection campaigns and comparative analysis under single-event upsets (SEUs). The framework supports interchangeable processor implementations and automated result analysis. Although the approach is architecture-agnostic, the implementation is evaluated using a custom RISC-V core. This evaluation enables a direct comparison of different fault mitigation techniques and therefore improves the reproducibility of experiments across processor variants and protection strategies. The following sections establish the space-radiation context and processor design constraints that make this comparison necessary.

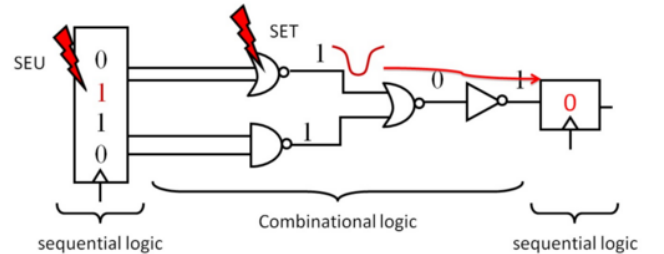


Fig. 1. Effect of a single event upset (SEU) on a circuit [8]

A. Space Environment

The space environment poses significant challenges for microelectronic devices, primarily due to the presence of high-energy ionizing radiation [6]. Space radiation consists of various types of energetic particles, including protons, electrons, and heavy ions, which originate from sources such as the Sun, cosmic rays, and trapped radiation belts around planetary bodies.

When these high-energy particles interact with microelectronic components, they can lead to various failure mechanisms. One of the most critical effects is the Single Event Upset, which occurs when an ionizing particle deposits enough energy to alter the state of a memory cell or register [6], [7]. SEUs can cause unintended changes in stored data, since these bit flips persist until overwritten or corrected and can lead to transient malfunctions in processors and other digital systems. In mission-critical space applications, such errors can have severe consequences, affecting system reliability and performance.

Mitigating these effects requires specialized design approaches, including error correction codes (ECC), radiation-hardened architectures, redundancy techniques, and software-based fault tolerance mechanisms [5], [6]. Figure 1 illustrates how an SEU affects circuit behavior.

a) Space Radiation Environment:

Three primary natural sources of radiation must be considered in space systems: galactic cosmic rays (GCRs), geomagnetically trapped particles, and solar energetic particles (SEPs) [9], [10].

Galactic cosmic rays originate outside the solar system and are believed to be produced by high-energy astrophysical events such as supernova explosions. They consist primarily of high-energy protons and heavy ions, with energies ranging from 10^8 to 10^{19} eV. Due to their extremely high energies, GCRs are highly penetrating and difficult to shield against, representing a continuous radiation background. Their flux is modulated by the solar cycle, peaking during solar minimum when heliospheric shielding is weakest [9].

Geomagnetically trapped particles form the Van Allen radiation belts, consisting of energetic protons and electrons confined by the Earth's magnetic field. The inner belt is

dominated by protons with energies up to several hundred MeV, while the outer belt is dominated by relativistic electrons. Radiation exposure strongly depends on orbital regime: low Earth orbit benefits from partial geomagnetic shielding, while medium Earth orbit intersects the most intense regions of the belts, and geostationary orbit is primarily exposed to outer-belt electrons and solar events [10].

A particularly important feature for low Earth orbit missions is the South Atlantic Anomaly, where the Earth's magnetic field is locally weakened. This allows energetic particles to penetrate to lower altitudes, making it the dominant source of radiation exposure for many LEO satellites [11].

Solar energetic particles are generated during solar flares and coronal mass ejections. These events produce intense fluxes of high-energy protons over short durations, typically lasting from hours to several days. Large solar particle events can dominate the radiation environment and pose a critical threat to spacecraft electronics [12].

b) *Physical Mechanisms of Radiation Effects:*

When high-energy particles interact with microelectronic components, they deposit energy in semiconductor materials through ionization processes. This interaction generates electron-hole pairs along the particle track, which are subsequently collected by electric fields within the device, producing transient currents [13].

The severity of this interaction is governed by the Linear Energy Transfer (LET), which quantifies the energy deposited per unit path length. If the deposited charge exceeds a technology-dependent threshold known as the critical charge (Q_{crit}), the state of a memory cell or logic node can be altered. As semiconductor technologies scale down, Q_{crit} decreases, making modern devices increasingly sensitive to radiation-induced disturbances [14].

Neutrons, although electrically neutral, can also induce radiation effects through nuclear reactions that generate secondary charged particles. These secondary particles then cause ionization similar to directly ionizing radiation, contributing significantly to error rates in certain orbital environments [9].

In SRAM-based devices such as FPGAs, the six-transistor memory cell is particularly vulnerable. A single particle strike can disturb the stored value of a cell, and due to the large amount of distributed configuration memory, the probability of such events becomes significant over mission lifetimes [15].

c) *Taxonomy of Single Event Effects:*

The interaction of radiation with semiconductor devices leads to a class of phenomena collectively known as Single Event Effects (SEEs). These effects can be broadly classified into non-destructive (soft errors) and potentially destructive (hard errors) [9], [13].

A Single Event Upset (SEU) is the most common soft error and represents a change of state in a memory element caused by a single particle strike. SEUs affect flip-flops, registers, and memory cells, and persist until corrected. They are a major concern in processors, as they can silently corrupt data or control flow [7], [14].

A Single Event Transient (SET) occurs in combinational logic, where a particle strike induces a temporary voltage pulse. If captured by a sequential element, this transient can

propagate as an error. Otherwise, it is typically masked by circuit timing or logic conditions [16].

Multi-Bit Upsets (MBUs) and Multi-Cell Upsets (MCUs) arise when a single radiation event affects multiple adjacent bits. These effects are increasingly common in modern technologies due to reduced feature sizes and tighter memory layouts, challenging traditional single-error correction schemes [14].

A Single Event Functional Interrupt (SEFI) occurs when radiation affects control logic or configuration memory, causing the system to enter an incorrect operational mode or halt. In complex devices such as processors and FPGAs, SEFIs can have system-wide consequences and often require a reset to recover [15].

In contrast, hard errors can lead to permanent or destructive damage. Single Event Latchup (SEL) is caused by the activation of parasitic structures in CMOS devices, resulting in a high-current state that persists until power is removed. If not mitigated, this can lead to thermal destruction of the device [9].

More severe destructive effects include Single Event Burnout (SEB) and Single Event Gate Rupture (SEGR), which primarily affect power devices and result in permanent failure due to structural damage within the transistor [9].

B. *Space-Grade Processors*

The on-board computer (OBC) executes mission-critical functions and must therefore operate under real-time constraints. In this context, real-time operation means meeting timing deadlines while maintaining predictable behavior. For example, attitude and orbit control require responsiveness, reacting quickly enough to maintain the satellite's trajectory, while also ensuring reliability through deterministic operation without unacceptable failures.

In order to fulfill these characteristics, state-of-the-art processors onboard satellites rely on simple architectures, often executing a single instruction per clock cycle and using simplified speculation or no speculative components at all [3], [17], [18], [19]. Limiting processor complexity in this sense can be beneficial, as the number of vulnerable state elements is reduced, making the reliability constraint more achievable. Non-deterministic behavior elements like caches and branch predictors do not affect architectural correctness in a fault-free run, but an SEU in these structures can perturb internal performance and timing in unpredictable ways. For example, an SEU in a branch predictor's memory may corrupt branch history, causing a flurry of mispredictions. Such faults often affect performance and timing rather than architectural correctness, although timing perturbations can still be mission-critical in hard real-time systems [20].

More worrisome is speculative out-of-order execution, where many instructions are in flight across a deep pipeline [21]. An SEU in the out-of-order machinery (e.g. the reorder buffer that tracks pending instructions, or the register renaming table) could corrupt commit ordering or data dependencies, potentially leading to incorrect program output or a fatal pipeline exception [20]. The complexity of recovering from mis-speculation adds difficulty in designing fault-tolerant control logic. In contrast, a simple in-order pipeline with no speculative issuance has a much smaller window of in-flight instructions, limiting the scope of an upset's immediate impact [22].

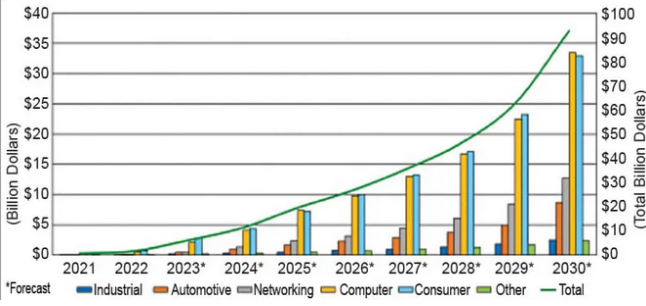


Fig. 2. Market revenues for all RISC-V SoCs by application [4]

C. RISC-V

RISC-V is an open-standard Instruction Set Architecture (ISA) that was developed at the University of California, Berkeley, beginning in 2010. Unlike proprietary ISAs such as x86 and ARM, RISC-V is designed with an emphasis on flexibility, modularity, and openness, allowing developers and researchers to tailor implementations to specific needs. RISC-V is structured around a small base ISA (either the 32-bit RV32I or 64-bit RV64I) that provides a minimal set of instructions for general-purpose integer computing. This base ISA can be extended with a number of optional standard extensions, such as floating-point arithmetic (F, D, Q), atomic operations (A), vector processing (V) among others as well as vendor-specific custom extensions [23].

One of the primary advantages of RISC-V is supplier independence. Since it is a non-proprietary ISA, there are no ISA licensing fees, reducing entry barriers for startups and academic institutions. This openness also fosters a growing community of developers and contributors, leading to rapid advancements and continuous innovation in the field. The modularity of RISC-V further enhances its appeal, as users can customize extensions and features to optimize performance for particular applications. For space systems, this openness is especially valuable because agencies and suppliers can define mission-specific extensions without long-term vendor lock-in. As a result, RISC-V has gained increasing traction in various industries, including artificial intelligence, automotive, embedded systems, and space applications [4].

Recent market analysis suggests a strong growth trajectory for RISC-V-based systems-on-chip (SoC) across multiple application domains (see Figure 2). At the same time, each implementation must be qualified for radiation tolerance, toolchain maturity, and software portability, making verification effort a key adoption challenge. In addition, the growing adoption of RISC-V in space applications is of particular interest due to its customizability, energy efficiency, and potential for implementing radiation-hardened designs [3].

D. Paper Structure

The remainder of this paper is organized as follows. Section II reviews prior mitigation techniques and fault-injection methodologies. Section III, Section IV, and Section V present the overall approach, the framework architecture, and the evaluated multi-cycle RV32I core. Section VI describes the benchmark and hardware setup and reports the results. Finally, Section VII summarizes the main findings and outlines future work.

II. STATE OF THE ART

This section summarizes state-of-the-art SEU mitigation techniques as well as discusses different approaches towards injection of artificial radiation-like faults into cores.

A. Fault Mitigation Techniques

a) Mitigation Techniques: Triple Modular Redundancy:

Triple Modular Redundancy (TMR) introduces redundant CPU components by triplicating them within the design. It operates under the assumption that at least two of three equivalent elements, when given the same signal, will return the correct value. To determine which elements produce the correct output, a voter system is employed, usually in the form of a multiplexer [24], see Figure 3 for a visual representation. This approach has proven to be robust over time, however, it is important to note that the voter system can become a single point of failure. SEUs can affect any type of sequential or combinational logic in a CPU; therefore, the voter component of a TMR architecture can also be impacted. In this case, the benefit of triplication could be lost. The key limitation of TMR is increased hardware overhead together with voter-related vulnerability. Wilson and Wirthlin showed the effectiveness of TMR on a RISC-V processor deployed on an FPGA, demonstrating that it reduced the neutron cross section by over 30 times [25].

Multiple methods for securing the TMR voter have been studied. In [24], the authors proposed applying the signal twice to a specially modified voter. The first signal, applied at time t_0 , is the original signal, while the signal at time $t_0 + \Delta t$ is its complement. The voter stores its verdict for the t_0 input and compares it with the verdict for the $t_0 + \Delta t$ input. When the two voter outputs are not the same, an error in the voter is detected. Errors in TMR are corrected through the Internal Configuration Access Port (ICAP) and Dynamic Partial Reconfiguration (DPR), typically implemented through a small-footprint processor. This approach, however, inevitably increases execution time.

The solution from [26] addresses the vulnerability of the TMR voter to SEUs by introducing a modified Triple Modular Redundancy with Disagreement Detection (MTMRDD). The proposed solution replicates voters and integrates disagreement detectors, using distributed decision-making to identify and mask faults in voters. If repeated disagreements occur, the system automatically switches to a spare voter, thus protecting against permanent and transient faults.

A similar approach to [26] in terms of voter replication is used in [27]. This paper presents a self-checking TMR voter specifically tailored for FPGA-based systems to mitigate SEUs. It employs multiple redundant voter circuits grouped into two sets that switch dynamically upon detecting dis-

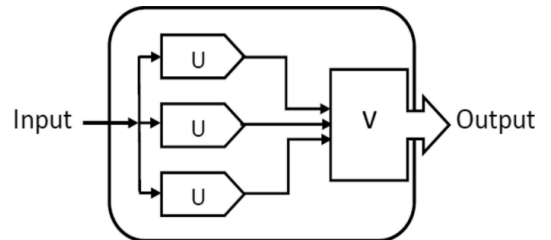


Fig. 3. Triple Modular Redundancy (TMR): three identical blocks evaluate the same logic function

agreements, thereby maintaining a fault-tolerant output. The proposed voter architecture integrates arbiter logic to control switching between redundant voters, significantly reducing the risk of voter failure compared to traditional simplex voters. Fault-injection analysis and reliability modeling indicate approximately a 26% improvement in reliability.

b) Error Correction Codes:

Error Correction Codes (ECC) are a family of techniques based on generating a unique sequence of bits that encodes information about the valid state of a bit array, for instance, a memory cell. The use of ECC can reduce the number of redundant components; in this case, it eliminates the need for a redundant memory [5]. A primary cost of ECC-based protection is additional memory and logic overhead for check bits and correction mechanisms.

Well-known space-industry architectures such as SPARC V7 with ERC-32 [18] and SPARC V8 with the LEON processor series [19] utilize ECC to protect memory. ERC-32 uses 32-bit SECDED ECC implemented in the Memory Controller (MEC) chip, where every 32-bit word fetched from memory carries extra check bits; single-bit memory errors are corrected on the fly and double-bit errors are detected. In LEON, the Level-1 caches (instruction and data) are protected by parity bits on each cache line. The larger Level-2 caches (present in newer LEON implementations such as LEON3FT/4FT) are typically protected with a Single Error Correction with Double Error Detection (SECDED) ECC code per line. Small arrays such as register files use triplication and voting. The space-grade RISC-V implementation NOEL-V [3] adopts the same ECC-utilization methodology as LEON.

In [28], RISC-V implementations, specifically the Rocket (in-order) [22] and BOOM (out-of-order) [21] cores, were analyzed in terms of the benefits of ECC incorporation. The authors implemented a configurable ECC memory solution (ECCmem), focusing primarily on parity, Single Error Correction (SEC), and SECDED methods. Their solution provided comprehensive memory protection across caches, branch predictors, and page-table walkers, and incorporated advanced techniques such as periodic scrubbing and correction buffering. The ECCmem implementation achieved significant improvements in fault tolerance with minimal performance impact and moderate hardware-resource overhead (approximately 5% logic overhead and approximately 40% additional RAM usage), highlighting the practicality of ECC for safety-critical aerospace and automotive applications.

c) Removal of Speculative Execution:

Speculative execution allows certain instructions to be performed in advance, which speeds up execution. Unfortunately, this additional logic adds non-deterministic behavior and increases design complexity, making such architectures more sensitive to SEUs. The same principle applies to branch predictors. Nevertheless, it is important to highlight that SEU-induced errors in predictors primarily affect performance, as SEU interference corrupts information about previously executed branches or instructions [20]. Removing speculative features generally reduces peak performance in exchange for improved determinism and robustness.

The constraints of real-time critical systems favor simpler cores. The failure of a computational unit controlling, for example, a satellite's altitude is highly likely to jeopardize

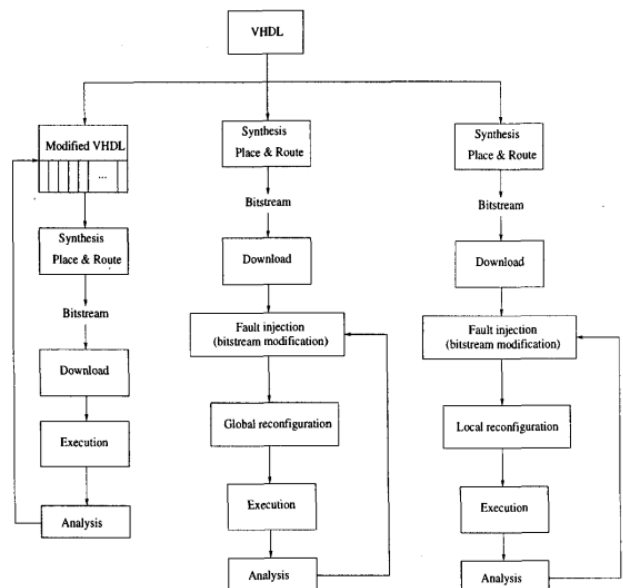


Fig. 4. FPGA reconfiguration schemes, [30]

an entire mission. SPARC V8 processors such as the LEON series [19] and SPARC V7 processors such as ERC-32 [18] employ in-order execution. ERC-32 utilizes a 4-stage in-order pipeline, while the more modern LEON series uses a 7-stage in-order pipeline (additionally, LEON4 supports dual-issue in-order execution and can dispatch two instructions at a time). These architectures deliberately sacrifice performance to comply with reliability requirements.

The most advanced approach is presented by the NOEL-V architecture [3], whose 7-stage in-order pipeline includes features such as branch prediction and a store buffer.

Regardless of the approach described above, current practice in safety-critical space processors generally favors in-order execution.

To evaluate how effectively these mitigation strategies perform in practice, controlled SEU injection and analysis methods are required. The following section reviews common injection methodologies and their trade-offs.

B. SEU Injection and Analysis

To validate the performance of an SEU-protected CPU, it is crucial to implement an injection mechanism that introduces artificial faults. Potential strategies can be divided according to how the fault is delivered to an FPGA. Each strategy is reviewed in the following subsections.

- Heavy Ion Radiation Test
- Reconfiguration techniques
- Hardware injection module

a) *Heavy Ion Radiation Test*: This technique is one of the most precise, as it applies a heavy-radiation beam directly to an FPGA board. The major benefit of this approach is that the upset can occur in any FPGA state, including states that are not directly targeted by other fault-injection schemes. The number of possible faults increases because radiation can induce more complex errors, for instance Single Event Transients, Multi-Cell Upsets, or Single Event Latch-up [29].

The LEON3 processor in [29] implements this methodology. The engineering team decided to incorporate the radiation campaign at the end, after previous soft-injection

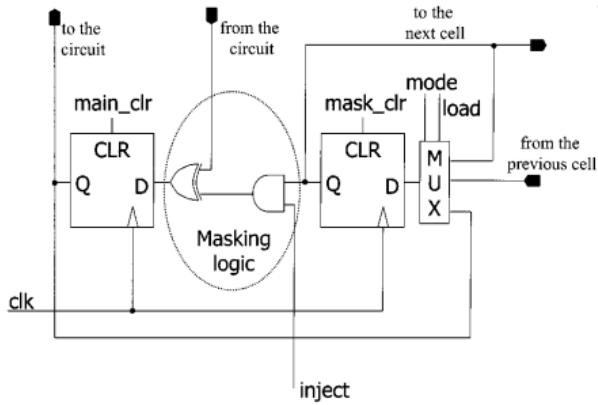


Fig. 5. Fault injection circuitry applied to flip-flop, [31]

schemes had validated the design. The core concept was to run two cores in parallel: one subjected to radiation and one not subjected to radiation. Comparing outputs from both cores provided a clear indicator of mitigation-design success or failure.

b) *Reconfiguration techniques:*

Reconfiguration techniques leverage configurable hardware such as FPGAs. Hardware Description Languages (HDLs), such as VHDL and Verilog, allow rapid synthesis of designs and repeated deployment to FPGAs. FPGA reconfiguration can be divided into two main types:

- Compile-Time-Reconfiguration (CTR)
- Run-Time-Reconfiguration (RTR)

CTR assumes a single system-wide configuration. This configuration is utilized to instantiate a design within an FPGA. In contrast, RTR dynamically changes the digital hardware several times. Depending on the reconfiguration scope, RTR can be defined as global RTR, when an entire design is modified, or local RTR, when only a specific area is impacted. Figure 4 presents workflows for each reconfiguration approach.

Reconfiguration techniques can be used both to correct an erroneous design and to deliberately inject faults [30]. During synthesis, a netlist is first created and then transformed into a bitstream that is uploaded to an FPGA. When CTR is used as an injection mechanism, each fault must be described in the design using HDL. This implies the need for multiple variations of the same project to cover the required fault scenarios. The circuit must be reconfigured whenever a new test case is considered.

The main benefit of RTR over CTR for error injection is that RTR avoids re-synthesis because all changes occur in the bitstream. Furthermore, the design does not require additional HDL source code to introduce the fault, as this occurs during bitstream modification; therefore, there is no permanent area overhead as in CTR. Nevertheless, a key implication of this solution, whether global or local, is that a strong understanding of the FPGA vendor's bitstream is required to apply changes to the desired areas.

c) *Hardware injection module:*

Additional hardware modules responsible for inducing faults into a system are particularly useful for large projects, where simulating the entire product in software is impractical. Moreover, injections can happen in a real-time, which allows running a large number of experiments, nonetheless

it introduces an additional complexity to the design [32]. In [31], [33], a fault-injection framework is presented that comprises a host machine that orchestrates which fault injections to administer and a circuit with an additional module that introduces errors. The authors of [33] consider single transient bit flips in circuit flip-flops, as shown in Figure 5. The decision of which bit to flip is made using a chain mask that associates each bit in a flip-flop with a corresponding mask element. This approach is reported to provide control over both fault timing and fault location. The work proposed in [33] is expanded in [34] to provide a mechanism for fault injection into a processor core and embedded memory. Nonetheless, the principal architecture of a host computer governing the injection process is maintained in [34]. To administer a fault inside a core, masking logic is applied to any relevant register. Embedded-memory-related faults are triggered through a special memory access mode during which incorrect data can replace correct data.

In [35], the OpenRISC 1200 microprocessor was subjected to fault injections administered by the host computer and delivered to specific processor areas through additional hardware within the design. As in the earlier papers, this work used hardware injectors programmed into the core logic using HDL and a software component running on the host machine to decide which faults to activate.

d) *Summary:*

Hardware injection requires a full access to core's internals without obstructing its work. An extensive survey of fault injection methods from [32] highlighted the risk of hardware injection methods having to stop the processor in order to apply a fault vector, which impacts the goal of measuring latencies. A software-based techniques assumes having a software running on an evaluated processor, where faults can be injected at the assembly level [32]. This requires no additional hardware, however the price is paid in more shallow level of faults penetration (restricted to the area accessible by software). The third approach is simulation-based injection, where a detailed HDL model of a processor is in use. This solution does not require to implement faults at the assembly level of a software running on-board and is able to model both permanent and transient faults. The main disadvantage of this direction is an extensive simulation time, that limits the amount of tests. An example of this technique applied to LEON3 core is presented in [36].

III. METHODOLOGY

This section provides a general overview of the employed methodology. First, the goals and overall architecture decisions are described, followed by the main technology choices. Subsequent sections provide deeper detail into individual aspects of the implementation, namely the fault injection system and evaluation RISC-V core.

A. Framework Goals and Overview

A framework was developed to systematically evaluate fault mitigation techniques for radiation-induced single event upsets (SEU) in RISC-V cores. Key design objectives include cost-effectiveness by avoiding expensive physical radiation testing, alongside a vendor-agnostic architecture

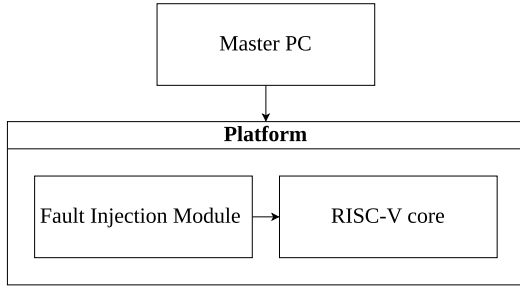


Fig. 6. Main components of the proposed framework

that facilitates reuse across different platforms and extends the potential impact of the work.

Four independent components comprise the overall architecture, as visualized in Figure 6: the RISC-V core under test, the fault injection layer, a platform component integrating the previous two, and a host application orchestrating the tests. This modular design remains independent of any specific core implementation, requiring only that the core conforms to a well-defined interface specification.

A comprehensive analysis of injection options (see Section II.B) led to the adoption of a custom hardware injection module that directly applies SEUs to processor components. This hybrid approach leverages a software layer to determine fault selection logic, while the actual fault injection occurs directly within the hardware, maximizing both flexibility and accuracy.

B. Technology Selection

Cost-effective physical fault injection necessitates a system-on-chip (SoC) architecture that integrates both a microprocessor and field programmable gate array (FPGA). Within this architecture, the microprocessor serves a dual purpose: providing a bridge to the host application (master PC) and managing fault injection into memory components. Individual SoC devices are referred to as “workers” throughout this work.

Alongside physical deployment, HDL simulation provides an alternative validation pathway, with platform logic implemented through simulator-specific tooling such as VHPI C code or Python.

The architecture deliberately promotes a language-agnostic approach with respect to the core implementation language, maintaining consistency across both unit/integration testing phases and hardware deployment. While vendor independence is partial, as proprietary synthesis software remains necessary for hardware deployment, the modular design supports extensions that enable integration of additional vendors and their respective synthesis/implementation tools.

Multiple Hardware Descriptive Languages (HDL) are supported:

- Verilog
- System Verilog
- VHDL

This flexibility derives from the use of vendor-specific synthesis and implementation tools capable of targeting different HDLs for hardware deployment, while the underlying build system itself is designed for extension to incorporate additional vendors and their FPGA deployment tools.

C. Framework verification/validation overview

Validation of the framework required an available RISC-V core suitable for fault injection experiments. Accordingly, a custom 32-bit integer-only RISC-V soft-core was implemented to minimize initial architectural complexity and enable focused evaluation. Critically, the core implements no mitigation techniques, as the primary objective centers on validating the framework infrastructure itself rather than demonstrating specific mitigation strategies.

Physical validation was performed using the Digilent Zybo Z7-20 board, which hosts a Xilinx SoC with an ARM microcontroller. For this platform, a dedicated Xilinx platform component was developed that integrates open-source solutions with Xilinx hardware and synthesis/implementation software to enable embedded deployment of the framework on the SoC. However, the overall design deliberately avoids vendor lock-in, as simulated platforms and support for alternative SoCs remain feasible and achievable through the modular architecture.

Comprehensive unit and integration test suites are available for both the evaluation RISC-V core and the fault injection module. Unit tests verify individual building blocks of the core and fault injection subsystem, while integration tests execute RISC-V programs on the fully assembled core within simulation to validate the correctness of the implemented instruction set architecture (ISA).

To support the language-agnostic design philosophy, traditional HDL testbenches were replaced with the Python-based package `cocotb` [37], which enables testbench development in Python and leverages its flexibility alongside an extensive ecosystem of libraries. Critically, `cocotb`’s runtime automatically abstracts differences between simulators for different HDL languages, allowing a single `cocotb` testbench to evaluate the same module across different HDL implementations, provided they maintain identical input/output port specifications. This unified testing approach substantially reduced overall development time and complexity.

IV. FAULT INJECTION FRAMEWORK

This section presents a detailed overview of the fault injection framework, diving into more detail on the design presented in the previous chapter. In particular, this section describes the fault injection workflow, the communication interface between the master PC and workers, as well as the architecture of each worker. The internal architecture of the worker is of particular interest as it highlights the interactions between the SoC platform and the components within it.

A. Overall Architecture

Figure 7 presents the fault injection workflow: The framework defines a worker: a single System-on-Chip (SoC) device that comprises of an FPGA and a microcontroller. The FPGA implements the evaluated core with hardware fault injection and control modules, while the microcontroller contains the software layer of the injection and control module. The RISC-V core inside each worker can either contain fault mitigations or not. In a similar manner it can be subjected to fault injections (affected worker) or not (unaffected worker): depending on the validation approach. The master PC is the host computer, which controls tests: it

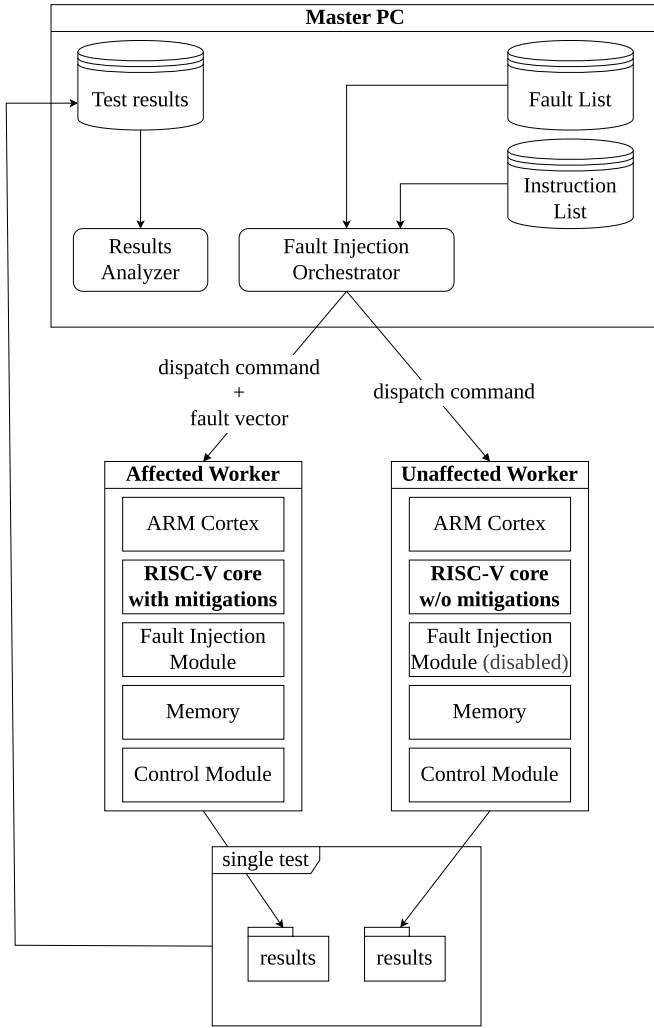


Fig. 7. Proposed fault injection workflow: the same faults are applied to an affected and unaffected worker and the results compared

selects programs to run, defines fault injection vectors and collects results. It is important to notice that the framework allows for fault injection tests both on the actual hardware as well as in simulation. In the later case, both the master PC application and simulation of the workers run on the same machine.

Similarly to the approach proposed by Civera et. al [34], the *Fault Injection Orchestrator* on the master PC can issue several commands to the worker:

- **START**: starts the core
- **STOP**: stops the core
- **STEP**: steps a single instruction forward
- **RESET**: resets the core
- **RESET <boot_addr>**: resets the core to a specific address
- **SET_PC <addr>**: specifies Program Counter
- **WRITE_WORD <addr> <value>**: writes a 32-bit word to a specified memory address
- **FAULT_REG <reg_idx> <mode> <mask>**: injects an error to a specified register
- **FAULT_MEM <addr> <mode> <mask>**: injects an error to a specified memory address
- **GET_PC**: reads the current Program Counter
- **GET_REG <reg_idx>**: reads value from a register

- **SET_REG <reg_idx> <value>**: sets the value of a register
- **READ_WORD <addr>**: reads a 32-bit from a specified memory address

The master PC orchestrates the fault injection experiments. Its responsibility is to accept the user's inputs and transfer them via UART to the worker in order to proceed with the program loaded to CPU's memory according to user's requests. It also verifies responses from workers.

Each worker consists of multiple components:

- **ARM Cortex**: hosts part of Fault Injection Module and Control Module; Acts as a proxy between the core and Master PC
- **implemented soft-core**: CPU modified to communicate with both Fault Injection Module and Control Module
- **Fault Injection Module**: injects faults inside selected regions of the core
- **Memory**: memory collecting both instructions and data
- **Control Module**: logic deciding whether to run or stop the core, it processes requests from Master PC

B. Single Worker Architecture

Due to a split of Fault Injection Module and Control Module in between software and hardware layers the following definitions apply:

- **Processing System Layer (microcontroller): PS**
- **Programmable Logic Layer (FPGA): PL**

Each worker integrates a soft-core inside PL and connects to the framework via components of it spread in-between PS and PL. The communication core-framework is established via AXI4 protocol, which currently is the state-of-the-art approach for communication between different peripherals of SoC in ARM based systems.

AXI4 distinguishes Master and a set of Slaves. Master has the right to initiate the transaction, decide with which Slave it wants to exchange information with. Figure 8 presents a high level hierarchy from AXI4 perspective, where ARM Cortex serves as the Master due to its direct connection to Master PC via UART. The Control Module resides partly on PS and partly on PL, therefore to correctly steer the core PS and PL elements of the Control Module exchange data via AXI and PL part of the Control Module serves as a Slave.

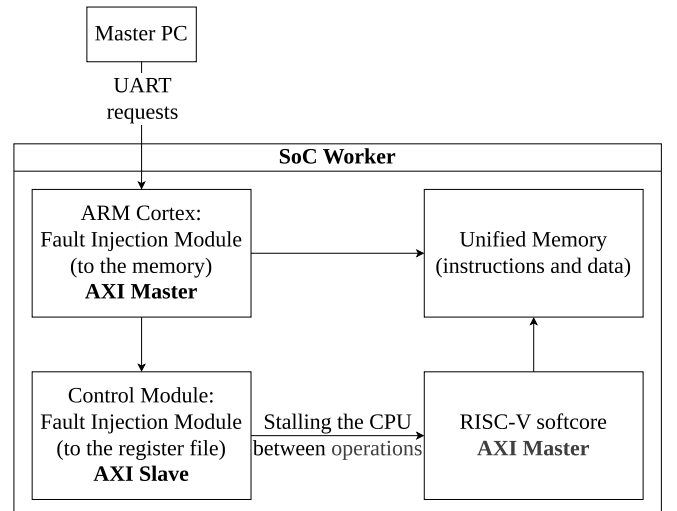


Fig. 8. Single worker

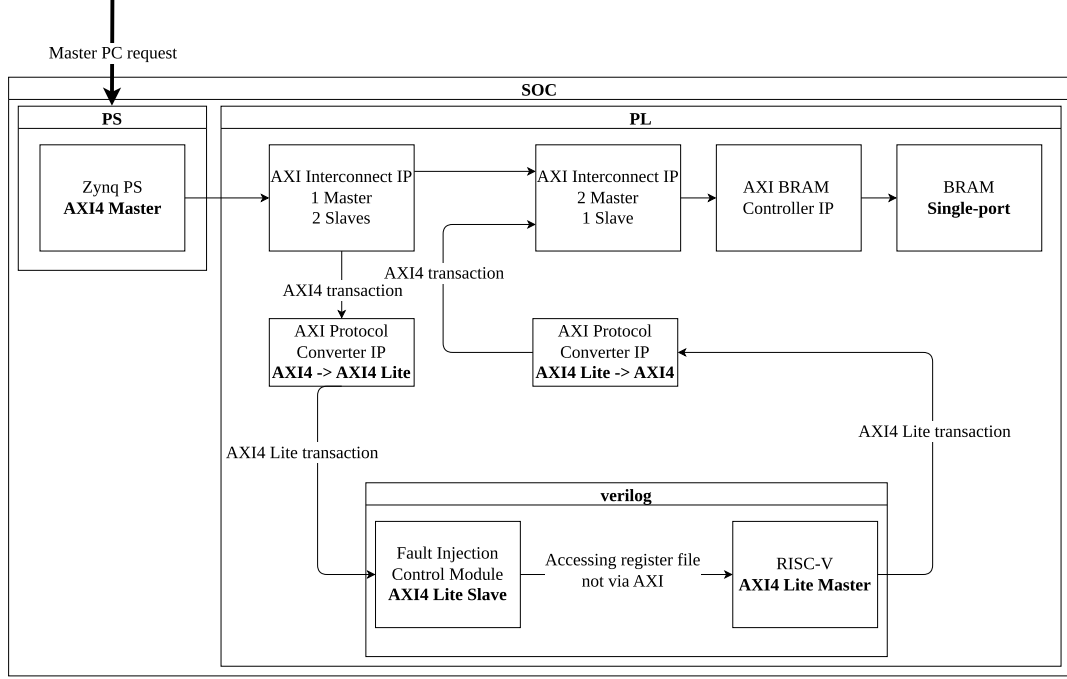


Fig. 9. Detailed block diagram of a single worker: The AXI4 protocol is used for communication between different components

Figure 9 dives deeper into the architecture of a worker. Visible IP Cores: AXI Interconnect IP and AXI Protocol Converter IP, allow to properly map all components residing on PL in order to be accessible for external (from PL perspective) read/write/inject operations. They also make it possible for the core to use Block RAM (BRAM) as its memory. The use of AXI Converter Protocol IP is crucial since PS operates using AXI4, while due to an ease of implementation and no need for extra bandwidth and features of AXI4 Fault Injection and Control Module on PL utilize AXI4Lite (less performant but easier to implement variant of the protocol). Soft-core (on Figure 9 marked as RISC-V) acts as AXI4Lite Master as it requests memory during instructions execution. Finally, AXI Interconnect IP creates address space in which all PL components are accessible.

V. RISC-V (RV32I) PROCESSOR FOR EVALUATION

To gain a deeper understanding of the RISC-V architecture and to systematically identify SEU vulnerabilities, the decision was made to implement a simple RV32I processor. The implementation followed official RISC-V unprivileged ISA and was backed up by a very similar implementation from HolyCore project [38]. This project was structured into several independent modules, each representing a key functional stage of the processor. In the sections that follow the design and implementation of each module, highlights the potential vulnerabilities to space-induced faults, and proposes corresponding mitigation strategies.

A. Memory Module

The memory subsystem is a critical component of the processor architecture, responsible for storing both program instructions and data during execution. The RISC-V implementation memory module was designed as Dual-Port Block RAM (BRAM) capable of handling 32-bit word-aligned accesses. It supports reading and writing operations using a simple interface controlled by address and enable signals.

This design was selected for both instruction memory as well as the data memory, in either case 4kB was picked as the size.

From a radiation-resilience standpoint, this module is particularly vulnerable due to the nature of on-chip memory structures in FPGAs, which are highly susceptible to Single Event Upsets (SEUs). SEUs may cause bit flips in stored data or even alter address bits, potentially leading to reads or writes at unintended memory locations- especially dangerous for instruction memory, where BRAM serves as a ROM memory, thus the write is meant to be always disabled [39]. These faults can silently corrupt program execution and are often difficult to detect without built-in fault detection mechanisms.

Mitigation strategies must therefore prioritize memory integrity. Relevant protection schemes involve memory scrubbing in addition to a method of error detection, either TMR or ECC- in a specific cases. Scrubbing involves periodically reading memory contents and rewriting them to their original state, thereby clearing any latent errors that may accumulate over time. Implementing a background scrubber can substantially increase system robustness without significantly impacting performance. In [39] scrubbing has been proven to work efficiently in conjunction with TMR, however the price of an enlarged area can be a discouraging factor for this approach. ECC is the technique very often associated with memories, however it is not preferred for protecting BRAMs against SEUs in space applications because it introduces single points of failure. Potentially vulnerable is write-enable line, that in this case represents a single point of failure. If such a signal is flipped by an SEU, it may corrupt the entire memory by enabling unintended writes — and ECC alone cannot prevent or correct this behavior. ECC does not inherently correct memory contents — only detects and corrects errors on read. It also requires specific memory configurations (e.g., 64-bit mode) [39] and fails to detect certain multi-bit errors like all-zeros or all-

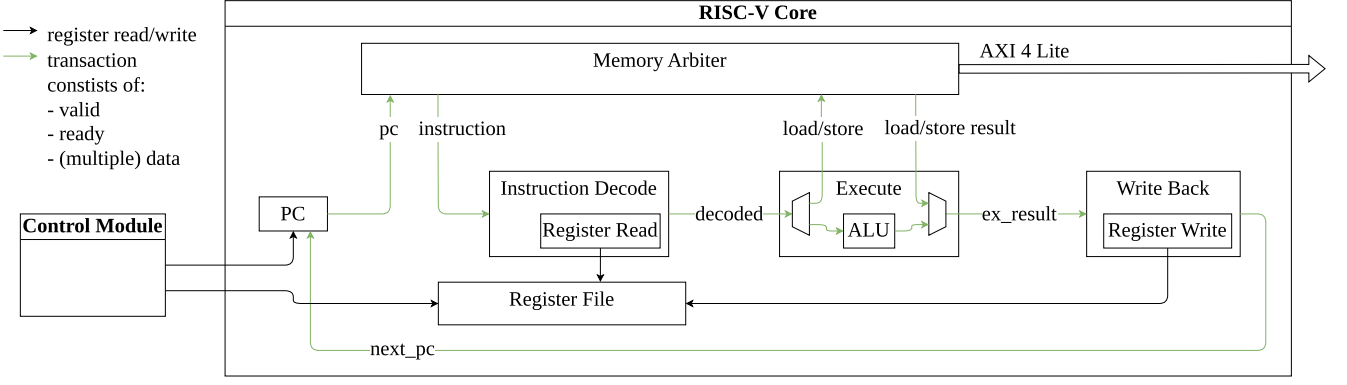


Fig. 10. Diagram of RV32I complete processor

ones upsets [29], [39]. As a result, ECC alone cannot prevent the accumulation of errors over time. In contrast, TMR combined with memory scrubbing provides more robust protection by masking faults and actively repairing memory content, making it the preferred solution for critical space systems. ECC may still be useful in non-critical or resource-constrained systems, especially when paired with duplication or scrubbing.

In conclusion, while the current memory implementation serves its functional role within the core, deploying it in a radiation-prone environment such as space necessitates substantial reinforcement through TMR and scrubbing or ECC, when the area constraints are very restrictive. These enhancements would ensure reliability and fault tolerance in mission-critical applications which we will uncover in future sections.

B. Register File

The register file is a central component of the RISC-V architecture, responsible for storing the processor's working data across instruction cycles. In the RV32I instruction set, there are 32 general-purpose registers, each 32 bits wide. The implementation of choice adheres to this standard and provides two simultaneous read ports and one write port, facilitating operand retrieval and result storage within a single clock cycle.

In the context of space environments, the register file represents a significant vulnerability to radiation-induced soft errors. As a dense collection of state-holding elements, it is highly susceptible to SEUs that can flip individual bits in the stored data. A bit flip in a source operand can lead to incorrect ALU computations, while corruption in a destination register may result in incorrect data being used by subsequent instructions. Furthermore, the control logic responsible for selecting read and write addresses can also be affected, potentially redirecting operations to unintended registers.

Given the importance of the register file to overall processor correctness, robust mitigation techniques are essential. A widely used method in radiation-tolerant designs is the incorporation of Error Detection and Correction (EDAC) schemes, particularly Hamming codes or more advanced SEC-DED (Single Error Correction, Double Error Detection) codes. These can be applied at the word level, protecting each 32-bit register independently [3]. Upon read, the EDAC decoder can correct single-bit upsets and flag

multi-bit anomalies, thereby improving system resilience without excessive hardware overhead.

A more lightweight compromise involves parity-based error detection [40]. By maintaining a parity bit for each register entry and verifying it during reads, the system can detect errors and trigger a software- or hardware-level recovery mechanism, such as instruction replay or exception handling.

Control logic that governs register address selection is equally critical. Redundancy in the decoding logic or simple voting mechanisms can help ensure that corrupted control signals do not lead to incorrect register accesses. Implementing range checks or verifying that writes do not target reserved or immutable registers (e.g., x0) can further protect against unintended side effects.

In summary, the register file is both a performance-critical and vulnerability-prone module in our RISC-V design. Its protection is paramount in any radiation-exposed deployment. Future iterations of this work will explore ECC as well as TMR implementations.

C. ALU Module

The Arithmetic Logic Unit (ALU) is the computational heart of the processor, responsible for executing arithmetic and logical operations dictated by the instruction set. In the RV32I processor implementation, the ALU supports a minimal yet representative subset of operations, including addition and bitwise logical operations. While the instruction set can be extended to include more complex functionality such as multiplication and division, we prioritized foundational instructions to streamline hardware design and verification.

The ALU operates on two 32-bit source operands and produces a 32-bit result. The operation performed is determined by a 4-bit control signal (ALU control), generated by the control unit based on the decoded instruction. The ALU also outputs a zero flag, which indicates whether the result of the operation is zero—an essential condition for implementing branch instructions such as BEQ.

In radiation-prone environments such as low-earth orbit or deep space, the ALU is vulnerable to Single Event Transients (SETs) and Single Event Upsets (SEUs). Transients in combinational logic can momentarily produce incorrect outputs due to particle strikes on logic gates, particularly in carry chains and multiplexers. Since the ALU is a core decision-making unit, even brief incorrect values can propagate through the system, causing errant memory writes

TABLE I
OVERVIEW OF SEU MITIGATION TECHNIQUES PER EACH DISCUSSED CPU MODULE

Module	TMR	Scrubbing	ECC/Parity
BRAM	Triplication of memory Voter system	Periodical scrubbing in conjunction with TMR	Useful only by itself for heavily area constrained designs
Register File	—	—	EDAC decoder Parity checks during reading ECC applied word-level
ALU	Triplication with voter system duplication of ALU spatial isolation	—	Use of One Shot Encodings
Control Module	Triplication of control signals decoders	—	ECC for critical instruction bits parity checks SEC-DED encoding on the fetched instruction

or corrupted program control flow. Additionally, SEUs can affect control signals such as alu control, leading to incorrect operations being selected.

To mitigate these risks, several strategies can be employed depending on system constraints such as area, power, and timing. One common approach is the implementation of Triple Modular Redundancy (TMR) at the ALU level. This involves instantiating three independent ALU units operating in parallel on the same inputs. A majority voting circuit then selects the most common output. TMR is highly effective against transient faults and single-bit errors but triples the hardware area and introduces additional delay through the voter logic.

Another approach is the use of redundant encoding of control signals. For instance, rather than using a compact 4-bit signal to encode ALU operations, one might use one-hot or error-detecting encodings, which allow detection—and potentially correction—of control line corruption. This reduces the likelihood of inadvertently selecting an unintended operation due to a single-bit flip.

Additionally, bit masking and result verification techniques can be introduced. For example, critical operations like zero comparison or signed overflow detection can be recomputed or checked against known invariants (e.g., $x - x = 0$). Lightweight duplication with comparison can also be effective for operations where full TMR is too resource-intensive: the same computation is performed twice, and the outputs are compared before acceptance.

Placement and routing constraints in the FPGA toolchain may also be leveraged to spatially isolate logic blocks, reducing the chance of multi-bit upsets affecting clustered ALU components. In some designs, guard gating is employed to disable unused logic paths dynamically, thereby reducing the active silicon vulnerable to particle strikes.

In summary, the ALU, though stateless in a single-cycle design, plays a critical role in data computation and program flow. As such, its protection is essential in any space-targeted implementation. Techniques such as TMR, encoded control signaling, and redundant computation provide viable paths toward a more resilient design. Future work includes integrating these approaches into our current FPGA-based core and evaluating their effectiveness through fault injection campaigns and post-synthesis analysis.

D. Control Unit

The control unit serves as the central interpreter of encoded instructions, transforming the raw bits fetched from memory into precise control signals that govern the behav-

ior of all subsequent modules. By examining the opcode, function3, and function7 fields in the instruction word, the unit determines whether an instruction requires a register write, a memory access, or a specific arithmetic or logical operation. Its design employs purely combinational logic arranged in a hierarchical decoding structure: a main decoder first categorizes opcodes into broad instruction types (load, store, arithmetic, etc.) and sets high-level control flags such as register write enable and memory write enable; then a secondary decoder refines the ALU operation code based on the function bits, ensuring that instructions like ADD, SUB, AND, and OR map to the correct arithmetic or logical functions.

In the radiation environment of space, the control unit is particularly susceptible to Single Event Upsets due to its high logic density and the critical role it plays in activating or inhibiting each functional block. A transient fault in an opcode bit can lead to misclassification of an instruction, for example treating a computation as a memory write, with potentially disastrous consequences for data integrity. Similarly, a flipped control bit in intermediate signals may disable essential operations, such as preventing a register from updating with a computed result, or inadvertently enable unauthorized writes or branches. Even the zero flag input, which guides conditional branches, can be corrupted, redirecting program flow in unpredictable ways and undermining the processor’s deterministic behavior.

Mitigation is therefore indispensable. Integrating Error-Correcting Codes (ECC) on critical instruction bits provides detection and correction of single-bit errors before decoding. Parity or SEC-DED encoding on the fetched instruction word ensures that corrupted instructions are identified and can trigger a safe recovery, such as pipeline flush or instruction replay. Complementing ECC, applying Triple Modular Redundancy (TMR) to core control signals—by instantiating three parallel decoders and voting on their outputs—suppresses single-event faults without necessitating additional pipeline stages.

In summary, protecting the control unit against SEUs relies on a combination of ECC for error detection and correction at the instruction interface and TMR for in-latch and combinational logic resilience. This dual approach preserves correct instruction interpretation, maintaining the processor’s functional integrity in the harsh radiation conditions of space. Table I aggregates the conclusions for this section.

VI. EVALUATION METHODOLOGY

This chapter discusses applied approach for evaluating the framework. Validation of the entire workflow was mandatory to prove that an arbitrary core with framework's infrastructure could be built, deployed on hardware after what it is able to respond to requests incoming from the Master PC: running given instructions, inject SEU fault, report back data.

The test sequence proceeds as follows:

- 1) The Master PC sends a command over UART to the PS.
- 2) The PS halts the RISC-V core via the Control Module.
- 3) Program instructions and initial register values are loaded through AXI.
- 4) The Master PC sets the boot address and send trigger to unstart the core.
- 5) The FIM schedules and performs the fault injection during core execution according to commands coming from the Master PC, relaid by Control Module.
- 6) The state of the subjected processor is monitored: register, Program Counter, memory, and sent back to the Master PC.

A. Hardware Setup

The verification of the framework was conducted using a custom multi-cycle RISC-V 32-bit soft-core that implemented Integer-only ISA (see Section V). The core was built and deployed on Digilent Zybo Z7-20 board. FPGA clock was set to 50MHz. The primary concern was to meet timing constraints on an FPGA fabric, however results collected in Table IV proven that the design had some margin represented primarily by metrics WNS and WPWS. WPS=+3.335ns indicates the time advantage of the slowest component at the clock path before the next clock tick, due to that theoretically the frequency of the clock could be increased to 60MHz.

The utilization of Zybo's FPGA fabric was very low: Table II and Table III, what highlight that Digilent's platform is suitable for the presented framework and could be used to experiment on far more complex cores than considered in this report.

B. Benchmark Selection

This chapter begins by discussing state-of-the-art benchmarks for processors with the focus on space-grade CPUs, followed by the selection of a benchmark for this project.

a) State-of-art benchmarks:

Selection of base case scenario is a key task for a successful fault injection campaign. The term base case refers to a task or set of tasks representative for a device under test. A base case could be a recognized benchmarking algorithm

TABLE II
SUMMARY OF FPGA SLICE LOGIC UTILIZATION

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs	4105	0	0	53200	7.72
└ LUT as Logic	4029	0	0	53200	7.57
└ LUT as Memory	76	0	0	17400	0.44
└ LUT as Distributed	0	0	—	—	—
RAM					
└ LUT as Shift Register	76	0	—	—	—
Slice Registers	3509	0	0	106400	3.30
└ Register as Flip Flop	3413	0	0	106400	3.21
└ Register as Latch	96	0	0	106400	0.09
F7 Muxes	484	0	0	26600	1.82
F8 Muxes	100	0	0	13300	0.75

TABLE III
SUMMARY OF FPGA MEMORY (BRAM) UTILIZATION

Site Type	Used	Fixed	Prohibited	Available	Util%
Block RAM Tile	2	0	0	140	1.43
└ RAMB36/FIFO	2	0	0	140	1.43
└ RAMB36E1 only	2	—	—	—	—
RAMB18	0	0	0	280	0.00

or a task that the device tackles during its mission lifespan. Typical benchmarks for validating an FPGA designs can be classified into software-defined and hardware-defined [41].

In terms of software-defined benchmarks one of the most popular benchmark is Dhrystone- invented in 1984 by Reinhold P. Weicker, it targets integer operations and intends to mimic typical CPU tasks. Dhrystone has been found useful in LEON3 validation on multiple occasions [40], [42], [43] as well as in soft-core processor presented in [25]. Whetstone is an another popular benchmark, it employs: floating point, branch assertion, fixed point, trigonometry, assignments and other non-linear mathematical (exp, log) operations. Some more case specific software tasks like matrix multiplications, sorting algorithms or I^2C are also utilized [34], [35] and they are found to be useful for SEU mitigation evaluation. CoreMark benchmark targets smaller microprocessors [41] and it implements operations like: matrix multiplications, list processing and state machine. CoreMark does not rely on external libraries calls. Moreover, it does not allow the data to be known during the compilation, therefore it mitigates Dhrystone issues of compiler optimizing the code before it even reaches the processor. Due to CoreMark enforcing specific rules for runs and reporting it avoids Dhrystone criticism of being more compiler benchmark than processor benchmark. RISC-V projects like CVA6 [44] utilizes both Dhrystone and CoreMark, while low-cost RISC-V for space application from [45] utilizes CoreMark, vector addition and CCSDS 123.

TABLE IV
DESIGN TIMING SUMMARY

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 3.335 ns	Worst Hold Slack (WHS): 0.024 ns	Worst Pulse Width Slack (WPWS): 9.020 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 9192	Total Number of Endpoints: 9192	Total Number of Endpoints: 3503

Hardware-based benchmarks are meant to stress flip-flops, routing, whereas software-based benchmarks interact with core's logic only indirectly. One of the most prominent suite of hardware benchmarks is the ITC'99 benchmark suite [41], [46]. It is a set of 22 fully-synthesizable HDL circuits that range from tiny 45-gate Finite State Machines (FMSs) to 30 k-gate control networks with thousands of flip-flops, all designed as single-clock, memory-free blocks for test-method research. Originally assembled for Design-for-Test and ATPG studies, the suite's open net-lists and accompanying stimulus/expected-response files make every instance a "self-checking peripheral" that can flag an upset with zero software effort. When ITC's circuits are placed alongside a soft-core in the same FPGA bit-stream, they exercise additional Look-up Tables (LUTs), flip-flops and routing resources that your processor alone may leave idle, so any fabric-level SEU-mitigation deployed must protect them too. This gives a wider, more realistic picture of residual vulnerability without extra coding [33] utilizes CoreMark.

Simulation-based benchmarks can benefit from having a detailed HDL model of a core under testing, without the need for an extra hardware. This approach presented in [36] implemented LEON3: one affected and one unaffected version for comparison. Authors of [36] used three different workloads in the fault injection campaign. These include a proportional integral derivative controller (PID), a bubble sort implementation (BSORT) and a hamming encoder (HAMMING).

b) Conclusions and Benchmark Selection:

During the development of the evaluation RISC-V processor described in Section V, we prepared a list of 27 RISC-V assembly programs specifically targeting the different types of instructions.

Those programs helped implementing the core and we decided to use them for fault injection validation because they all combined test every building component of RISC-V ISA.

c) Results:

All test programs ran successfully both in simulation and on the worker SoC when executed without injecting any faults. Since currently no mitigations are implemented in the core, injecting any fault (in the register file or memory) directly leads to data corruption.

VII. CONCLUSION

The aim of this project was to design a systematic evaluation framework of RISC-V single event upset (SEU) mitigation techniques. The product of this challenge is a generic framework capable of injecting synthetic SEU faults into certain areas of an arbitrary RISC-V core implemented on an FPGA fabric. The framework provides researchers with a set of convenient open-source tools for verifying the progress of core's development as well as a modifiable build system to target different vendors.

The master PC component of the system allows for interactive testing of a selected soft-core.

A. Challenges

An initial challenge encountered during this project was to reconcile FPGA vendor tools necessary for synthesis and implementation with the source control system. An integra-

tion of both open-source simulators with aforementioned proprietary software into Python-based build system took a significant of time.

The compile time for FPGA designs when deploying it onto hardware took very long time even though the design by itself consumes a small percentage of FPGA resources. This issue was never solved.

Finally, as this project used Xilinx FPGA device we were tied to AMD's development environment. Not only did it prove to be difficult for the source control system but from time to time yielded unexplainable behaviors: in order to update `xsa` file AMD Vitis project had to be deleted and created from scratch, otherwise changes were not being applied.

B. Future Work

Based on our experience and the overall results we have identified multiple potential points for the future work:

- creation of standardized interface that will connect the framework with any given soft-core; Creation of development guidelines listing what type of ports a particular core should expose to the framework.
- integration of other open-source RISC-V implementations to demonstrate the robustness of the solution.
- implementation of an internal timer and memory region to save print calls proceeded by CoreMark integration.
- pipelining the provided RISC-V multicycle core
- incorporating fault mitigation techniques into provided RISC-V core and a systematic comparison between mitigated and unmitigated scenarios.

REFERENCES

- [1] G. Furano, S. Di Mascio, A. Menicucci, and C. Monteleone, "A European Roadmap to Leverage RISC-V in Space Applications," in *2022 IEEE Aerospace Conference (AERO)*, Mar. 2022, pp. 1–7. doi: 10.1109/AERO53065.2022.9843361.
- [2] Q. Chen, Y. Liu, Z. Wu, and J. Liao, "A Single Event Effect Simulation Method for RISC-V Processor," in *2021 IEEE 15th International Conference on Anti-counterfeiting, Security, and Identification (ASID)*, Oct. 2021, pp. 106–110. doi: 10.1109/ASID52932.2021.9651696.
- [3] N.-J. Wessman *et al.*, "De-RISC: the First RISC-V Space-Grade Platform for Safety-Critical Systems," in *2021 IEEE Space Computing Conference (SCC)*, Aug. 2021, pp. 17–26. doi: 10.1109/SCC49971.2021.00010.
- [4] S. Kharoo, "RISC-V Market Trends And Predictions Till 2030." Accessed: Oct. 20, 2025. [Online]. Available: <https://www.electronicsforu.com/technology-trends/risc-v-market-trends-predictions-till-2030>
- [5] A. Dörflinger, B. Kleinbeck, M. Albers, H. Michalik, and M. Moya, "A Framework for Fault Tolerance in RISC-V," in *2022 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCom/CyberSciTech)*, Sept. 2022, pp. 1–8. doi: 10.1109/DASC/PiCom/CBDCom/Cy55231.2022.9927800.
- [6] European Cooperation for Space Standardization, "Engineering techniques for radiation effects mitigation in ASICs and FPGAs handbook," technical report ECSS-E-HB-20-40A, Oct. 2023. Accessed: Oct. 20, 2025. [Online]. Available: <https://ecss.nl/home/ecss-e-hb-20-40a-engineering-techniques-for-radiation-effects-mitigation-in-asics-and-fpgas-handbook/>
- [7] J. W. Howard and D. M. Hardage, "Spacecraft Environments Interactions: Space Radiation and Its Effects on Electronic System," Technical Publication (TP) NASA/TP-1999-209373, July 1999. Ac-

- cessed: Oct. 20, 2025. [Online]. Available: <https://ntrs.nasa.gov/citations/19990116210>
- [8] J. R. Azambuja, F. Kastensmidt, and J. Becker, *Hybrid Fault Tolerance Techniques to Detect Transient Faults in Embedded Processors*. Cham: Springer International Publishing, 2014. doi: 10.1007/978-3-319-06340-9.
- [9] J. Srouf and J. McGarrity, "Radiation effects on microelectronics in space," *Proceedings of the IEEE*, vol. 76, no. 11, pp. 1443–1469, 1988, doi: 10.1109/5.90114.
- [10] European Cooperation for Space Standardization, "Space Environment Standard ECSS-E-ST-10-04C." 2023.
- [11] C. Dyer and P. Truscott, "The South Atlantic Anomaly: A Review," *IEEE Transactions on Nuclear Science*, vol. 54, no. 4, pp. 1018–1024, 2007, doi: 10.1109/TNS.2007.903932.
- [12] M. Xapsos, C. Stauffer, G. Gee, J. Barth, E. Stassinopoulos, and R. McGuire, "Model for solar proton risk assessment," *IEEE Transactions on Nuclear Science*, vol. 51, no. 6, pp. 3394–3398, 2004, doi: 10.1109/TNS.2004.839159.
- [13] G. C. Messenger, "Single-Event Phenomena," *IEEE Transactions on Nuclear Science*, vol. 44, no. 6, pp. 2150–2160, 1997, doi: 10.1109/23.659129.
- [14] R. Baumann, "Soft Errors in Advanced Semiconductor Devices—Part I: The Three Radiation Sources," *IEEE Transactions on Device and Materials Reliability*, vol. 5, no. 3, pp. 305–316, 2005, doi: 10.1109/TDMR.2005.853449.
- [15] H. Quinn, "Radiation effects in reconfigurable FPGAs," *Semiconductor Science and Technology*, vol. 32, no. 4, p. 44001, Mar. 2017, doi: 10.1088/1361-6641/aa57f6.
- [16] V. Ferlet-Cavrois, L. W. Massengill, and P. Gouker, "Single Event Transients in Digital CMOS—A Review," *IEEE Transactions on Nuclear Science*, vol. 60, no. 3, pp. 1767–1790, 2013, doi: 10.1109/TNS.2013.2255624.
- [17] S. Di Mascio, A. Menicucci, E. Gill, and C. Monteleone, "Extending the NOEL-V Platform with a RISC-V Vector Processor for Space Applications," *Journal of Aerospace Information Systems*, vol. 20, no. 9, pp. 565–574, Sept. 2023, doi: 10.2514/1.1011097.
- [18] V. Stachetti, J. Gaisler, G. Goller, and C. Le Gargasson, "32-bit processing unit for embedded space flight applications," *IEEE Transactions on Nuclear Science*, vol. 43, no. 3, pp. 873–878, June 1996, doi: 10.1109/23.510727.
- [19] J. Andersson, M. Hjorth, F. Johansson, and S. Habinc, "LEON Processor Devices for Space Missions: First 20 Years of LEON in Space," in *2017 6th International Conference on Space Mission Challenges for Information Technology (SMC-IT)*, Sept. 2017, pp. 136–141. doi: 10.1109/SMC-IT.2017.31.
- [20] J. Mach, L. Kohútka, and P. Čičák, "In-Pipeline Processor Protection against Soft Errors," *Journal of Low Power Electronics and Applications*, vol. 13, no. 2, p. 33, June 2023, doi: 10.3390/jlpeal3020033.
- [21] C. Celio, P.-F. Chiu, B. Nikolic, D. A. Patterson, and K. Asanović, "BOOMv2: an open-source out-of-order RISC-V core," Boston, MA, USA, Oct. 2017. [Online]. Available: <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2017/EECS-2017-157.html>
- [22] K. Asanović *et al.*, "The Rocket Chip Generator," technical report UCB/EECS-2016-17, Apr. 2016. Accessed: Oct. 20, 2025. [Online]. Available: <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-17.html>
- [23] RISC-V Foundation, "The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 20250508." [Online]. Available: https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf
- [24] D. G. Mahmoud *et al.*, "Fault secure FPGA-based TMR voter," in *2018 7th Mediterranean Conference on Embedded Computing (MECO)*, June 2018, pp. 1–4. doi: 10.1109/MECO.2018.8406016.
- [25] A. E. Wilson and M. Wirthlin, "Neutron Radiation Testing of Fault Tolerant RISC-V Soft Processor on Xilinx SRAM-based FPGAs," in *2019 IEEE Space Computing Conference (SCC)*, Pasadena, CA, USA: IEEE, July 2019, pp. 25–32. doi: 10.1109/SpaceComp.2019.00008.
- [26] A. Patooghy, S. G. Miremadi, A. Javadtalab, M. Fazeli, and N. Farazmand, "A Solution to Single Point of Failure Using Voter Replication and Disagreement Detection," in *2006 2nd IEEE International Symposium on Dependable, Autonomic and Secure Computing*, Sept. 2006, pp. 171–176. doi: 10.1109/DASC.2006.15.
- [27] U. Afzaal and J.-A. Lee, "A Self-Checking TMR Voter for Increased Reliability Consensus Voting in FPGAs," *IEEE Transactions on Nuclear Science*, vol. 65, no. 5, pp. 1133–1139, May 2018, doi: 10.1109/TNS.2018.2824821.
- [28] A. Dörflinger, Y. Guan, S. Michalik, S. Michalik, J. Naghmouchi, and H. Michalik, "ECC Memory for Fault Tolerant RISC-V Processors," in *Architecture of Computing Systems – ARCS 2020*, A. Brinkmann, W. Karl, S. Lankes, S. Tomforde, T. Pionteck, and C. Trinitis, Eds., Cham: Springer International Publishing, 2020, pp. 44–55. doi: 10.1007/978-3-030-52794-5_4.
- [29] M. J. Wirthlin, A. M. Keller, C. McCloskey, P. Ridd, D. Lee, and J. Draper, "SEU Mitigation and Validation of the LEON3 Soft Processor Using Triple Modular Redundancy for Space Processing," in *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, in FPGA '16. New York, NY, USA: Association for Computing Machinery, Feb. 2016, pp. 205–214. doi: 10.1145/2847263.2847278.
- [30] L. Antoni, R. Leveugle, and B. Feher, "Using run-time reconfiguration for fault injection in hardware prototypes," in *Proceedings IEEE International Symposium on Defect and Fault Tolerance in VLSI Systems*, Oct. 2000, pp. 405–413. doi: 10.1109/DFTVS.2000.887181.
- [31] P. Civera, L. Macchiarulo, M. Rebaudengo, M. Reorda, and M. Violante, "Exploiting circuit emulation for fast hardness evaluation," *IEEE Transactions on Nuclear Science*, vol. 48, no. 6, pp. 2210–2216, Dec. 2001, doi: 10.1109/23.983197.
- [32] H. Ziade, R. Ayoubi, and R. Velazco, "A survey on fault injection techniques," *The international Arab journal of information technology*, vol. 1, no. 2, pp. 171–186, Jan. 2004, [Online]. Available: <https://hal.science/hal-00105562>
- [33] P. Civera, L. Macchiarulo, M. Rebaudengo, M. Reorda, and A. Violante, "Exploiting FPGA for accelerating fault injection experiments," in *Proceedings Seventh International On-Line Testing Workshop*, July 2001, pp. 9–13. doi: 10.1109/OLT.2001.937810.
- [34] P. Civera, L. Macchiarulo, M. Rebaudengo, M. Reorda, and M. Violante, "FPGA-based fault injection for microprocessor systems," in *Proceedings 10th Asian Test Symposium*, Nov. 2001, pp. 304–309. doi: 10.1109/ATS.2001.990301.
- [35] M. Shokrolah-Shirazi and S. G. Miremadi, "FPGA-Based Fault Injection into Synthesizable Verilog HDL Models," in *2008 Second International Conference on Secure System Integration and Reliability Improvement*, July 2008, pp. 143–149. doi: 10.1109/SSIRI.2008.47.
- [36] R. Travessini, P. R. C. Villa, F. L. Vargas, and E. A. Bezerra, "Processor core profiling for SEU effect analysis," in *2018 IEEE 19th Latin-American Test Symposium (LATS)*, Mar. 2018, pp. 1–6. doi: 10.1109/LATW.2018.8347235.
- [37] cocotb contributors, "cocotb." Accessed: Mar. 25, 2026. [Online]. Available: <https://www.cocotb.org/>
- [38] H. Babin-Riby, "HOLY CORE COURSE Learn how to build an RV32I core and deploy it on FPGA." Accessed: Oct. 20, 2025. [Online]. Available: https://github.com/0BAB1/HOLY_CORE_COURSE
- [39] N. Rollins, M. Fuller, and M. J. Wirthlin, "A Comparison of fault-tolerant memories in SRAM-based FPGAs," in *2010 IEEE Aerospace Conference*, Mar. 2010, pp. 1–12. doi: 10.1109/AERO.2010.5446661.
- [40] J. Gaisler, "A portable and fault-tolerant microprocessor based on the SPARC v8 architecture," in *Proceedings International Conference on Dependable Systems and Networks*, June 2002, pp. 409–415. doi: 10.1109/DSN.2002.1028926.
- [41] H. Quinn *et al.*, "Using Benchmarks for Radiation Testing of Microprocessors and FPGAs," *IEEE Transactions on Nuclear Science*, vol. 62, no. 6, pp. 2547–2554, Dec. 2015, doi: 10.1109/TNS.2015.2498313.
- [42] K. Marcinek, A. W. Luczyk, and W. A. Pleskacz, "Enhanced LEON3 core for superscalar processing," in *2009 12th International Symposium on Design and Diagnostics of Electronic Circuits & Systems*, Apr. 2009, pp. 238–241. doi: 10.1109/DDECS.2009.5012137.
- [43] A. Kchaou, W. El Hadj Youssef, and R. Tourki, "Performance evaluation of multicore LEON3 processor," in *2015 World Symposium on*

Computer Networks and Information Security (WSCNIS), Sept. 2015, pp. 1–4. doi: 10.1109/WSCNIS.2015.7368277.

- [44] F. Zaruba and L. Benini, “The Cost of Application-Class Processing: Energy and Performance Analysis of a Linux-Ready 1.7-GHz 64-Bit RISC-V Core in 22-nm FDSOI Technology,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 11, pp. 2629–2640, Nov. 2019, doi: 10.1109/TVLSI.2019.2926114.
- [45] D. A. Santos, L. M. Luza, C. A. Zeferino, L. Dilillo, and D. R. Melo, “A Low-Cost Fault-Tolerant RISC-V Processor for Space Systems,” in *2020 15th Design & Technology of Integrated Systems in Nanoscale Era (DTIS)*, Apr. 2020, pp. 1–5. doi: 10.1109/DTIS48698.2020.9081185.
- [46] F. Corno, M. Reorda, and G. Squillero, “RT-level ITC’99 benchmarks and first ATPG results,” *IEEE Design & Test of Computers*, vol. 17, no. 3, pp. 44–53, July 2000, doi: 10.1109/54.867894.

APPENDIX A

LEGACY

This section documents the artifacts and resources available for future research and development.

All project code is maintained in a public git repository¹, including the fault-injection framework, simulation infrastructure, and analysis tools. The repository contains detailed markdown documentation with setup instructions, usage examples, and guidelines for extending the framework to support additional processor implementations and mitigation strategies. This enables future students and researchers to build upon the work without reimplementing core infrastructure.

¹<https://github.com/Peak-Embedded-Projects/RISCV-Research-Project>